CS4475 Fall 2026 Lab Report
Diego A Guerrero Submitted on 10/14/2026
daguerrero9@miners.utep.edu

PA2: Priority-based Scheduler for xv6

Task 1. (15 pts) Implement the getprocs() system call and test it using the provided ps command.

I defined struct pstat in kernel/pstat.h and added getprocs to the syscall interface through kernel/defs.h, kernel/sysproc.c, kernel/syscall.h, kernel/syscall.c, user/user.h, and user/usys.pl. The implementation in kernel/proc.c walks the process table, copies active process records into the caller's array, and returns the count. I added user/ps.c to the Makefile. The source uses xv6's supported %lu format for the memory-size field.

```
$ ps
pid  state     size   ppid  name
1    sleeping  16384  0     init
2    sleeping  20480  1     sh
3    running   16384  2     ps
```

The three active rows and their parent IDs confirm that ps received process snapshots from getprocs. This stage is recorded in commit e5d4a10.

Task 2. (15 pts) Modify the provided ps command to print the priority of each process.

I added a base priority to struct proc, initialized it to 0 in allocproc, and copied it to children in kfork. MINPRIORITY and MAXPRIORITY define the allowed range 0 through 49. The getpriority and setpriority system calls were registered in the kernel syscall table and user stubs. getprocs now copies priority into struct pstat, and ps prints it. I also added prioritytest and pexec to the user programs.

```
$ prioritytest
initial priority: 0
child inherited priority: 49
child changed priority: 7
parent priority after child exits: 49
prioritytest: PASS
```

```
$ pexec 10 ps
pid  state     size   priority  ppid  name
1    sleeping  16384  0         0     init
2    sleeping  20480  0         1     sh
3    sleeping  16384  10        2     pexec
4    running   16384  10        3     ps
```

With pexec 10 ps, both pexec and its ps child appeared at priority 10. With pexec 49 ps, both appeared at 49. pexec 50 ps was rejected with 'priority must be between 0 and 49.' These results verify initialization, validation, inheritance, and the ps column. Commit: 9d2e1cc.

Task 3. (15 pts) Add a readytime field to struct proc, initialize it correctly, and modify ps to print a process's age.

I added readytime to struct proc and struct pstat. The make_runnable helper records the current tick when a process enters RUNNABLE from userinit, kfork, yield, wakeup, or kkill. The ps command samples uptime once and prints uptime minus readytime for RUNNABLE processes; it prints a dash for other states.

```
$ matmul 100 &; ps
pid  state     size   age  priority  ppid  name
1    sleeping  16384  -    0         0     init
2    sleeping  20480  -    0         1     sh
3    running   16384  -    0         2     ps
5    runnable  16384  0    0         1     matmul
```

The runnable matmul process had age 0 because it entered the ready queue in the same clock tick as the ps sample. Commit: be3ab28.

Task 4. (15 pts) Implement a priority-based scheduler.

kernel/param.h defines SCHED_RR, SCHED_PRIORITY, and SCHED_POLICY for compile-time selection. In kernel/proc.c, run_runnable keeps the shared context-switch sequence; the round-robin branch scans the process table in order, while the priority branch selects the highest base-priority RUNNABLE process. The Makefile uses one QEMU CPU so the comparison runs on a single scheduler. user/schedtest.c creates a priority-5 child before a priority-30 child, synchronizes their release with pipes, and records their order.

SCHED_PRIORITY: schedtest -> H then L; PASS
SCHED_RR: schedtest -> L then H; PASS

The test showed that selection policy changed which child ran first. This task clarified why process selection and context switching must keep the same lock protocol. A challenge was making both children ready before comparing order; the pipe barrier made that comparison controlled. Both policies built and booted. Commit: 452cd9e.

Task 5. (15 pts) Add aging to your priority-based scheduler.
I added SCHED_AGING and capped effective priority at 99. For a RUNNABLE process, effective priority is min(99, base priority + current tick - readytime). The wait begins when make_runnable records readytime. The scheduler compares effective priorities while holding each candidate's process lock. The plain-priority and round-robin modes remain available. The committed policy is SCHED_AGING.

agingtest released a short priority-0 child and a priority-49 CPU-bound child together. The long child ran for 120 ticks. Response is the first run tick minus the release tick; turnaround is completion minus release. Both tests passed under each policy, and schedtest still reported H then L.

Policy	**Low response**	**Average response**	**Average turnaround**
Plain priority	120 ticks	60 ticks	120 ticks
Aging	49 ticks	24 ticks	84 ticks

In this controlled run, aging let the short job start at tick 49 rather than tick 120. Average response fell from 60.0 to 24.5 ticks and average turnaround from 120.0 to 84.5 ticks. I learned how a ready-queue timestamp can express waiting time without updating an age counter every tick. The main difficulty was producing a fair comparison, so the same test and workload were run after rebuilding each policy. These are single-run measurements. Commit: fa48b46.

Task 6. (5 pts) Summary
I learned how a new xv6 system call connects the user declaration and stub to the kernel dispatcher and process table. I also learned how priority inheritance, RUNNABLE transitions, and timer ticks interact with scheduling. Testing both scheduler policies with synchronized children made the scheduling decision visible and measurable.

Extra Credit Task (10 points).
I added getpriority and setpriority as built-in commands in user/sh.c. They run before the shell forks an external command, allowing setpriority to change the current shell process. A decimal parser accepts one value from 0 through 49 and rejects invalid input. After setpriority 12, getpriority printed 12 and ps showed both sh and its child ps at priority 12. Inputs 50 and abc were rejected and the shell remained at 12. Commit: f933418.

$ getpriority -> priority: 0
$ setpriority 12 -> priority set to 12
$ ps -> sh: 12, ps: 12
$ setpriority 50 -> usage: setpriority <0-49>
$ setpriority abc -> usage: setpriority <0-49>

To measure an interactive-style short job under contention, I added agingtest inherit. The short child keeps the shell's priority while the competing long child uses priority 49 for 120 ticks. At shell priority 0, the short child's response was 49 ticks; after setpriority 40, it was 9 ticks. Both runs passed. This controlled comparison shows the benefit of raising the shell's priority for a short command in this workload. Commit: da70f76.

Resources: Fall 2026 PA2 assignment handout and report template; the xv6 source in the course repository; the supplied ps.c and pexec.c examples. All reported measurements came from the VMware Ubuntu VM.

Assistance disclosure: I used OpenAI ChatGPT to plan implementation steps, draft terminal commands, review output errors, and resolve those errors. I entered and ran the implementation and tests in the Ubuntu VM, checked the resulting diffs and outputs, and committed the changes on pa2.